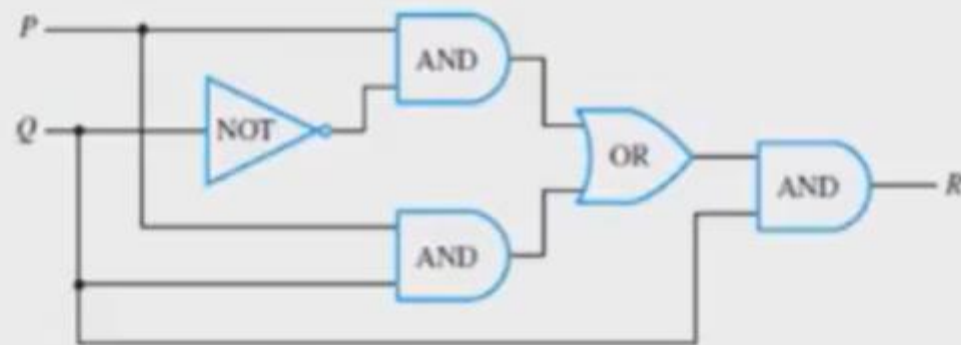


Equivalent Circuits

Following is the circuit representations of the statement

$$[(P \wedge \sim Q) \vee (P \wedge Q)] \wedge Q$$

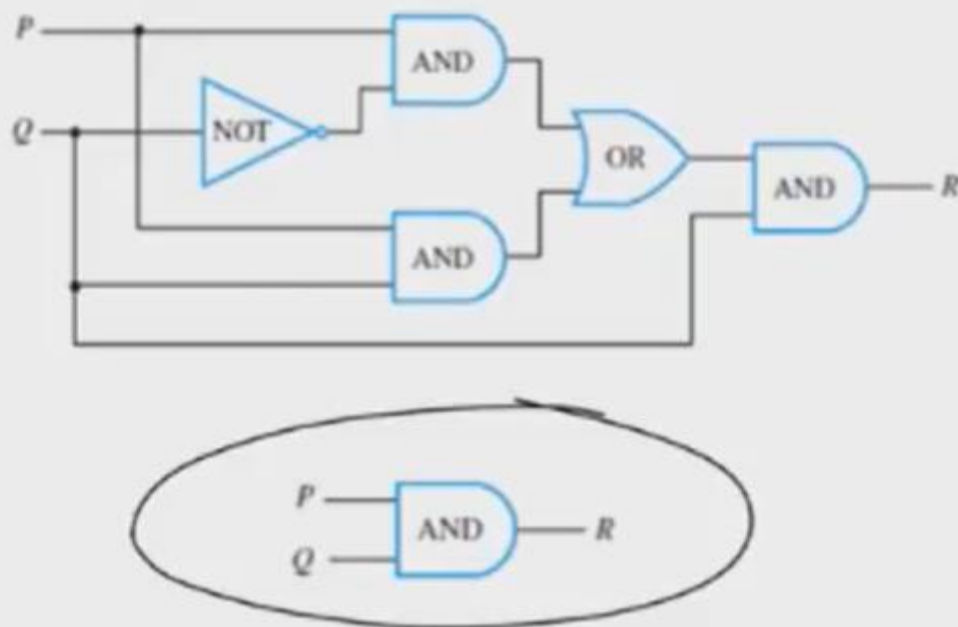


Equivalent Circuits

$$\begin{aligned} \text{LHS} &= [(P \wedge \sim Q) \vee (P \wedge Q)] \wedge Q \\ &\equiv (P \wedge (\sim Q \vee Q)) \wedge Q \quad ; \text{by the distributive law} \\ &\equiv (P \wedge (Q \vee \sim Q)) \wedge Q \quad ; \text{by the commutative law for } \vee \\ &\equiv (P \wedge \mathbf{t}) \wedge Q \quad ; \text{by the negation law} \\ &\equiv P \wedge Q \quad ; \text{by the identity law.} \end{aligned}$$

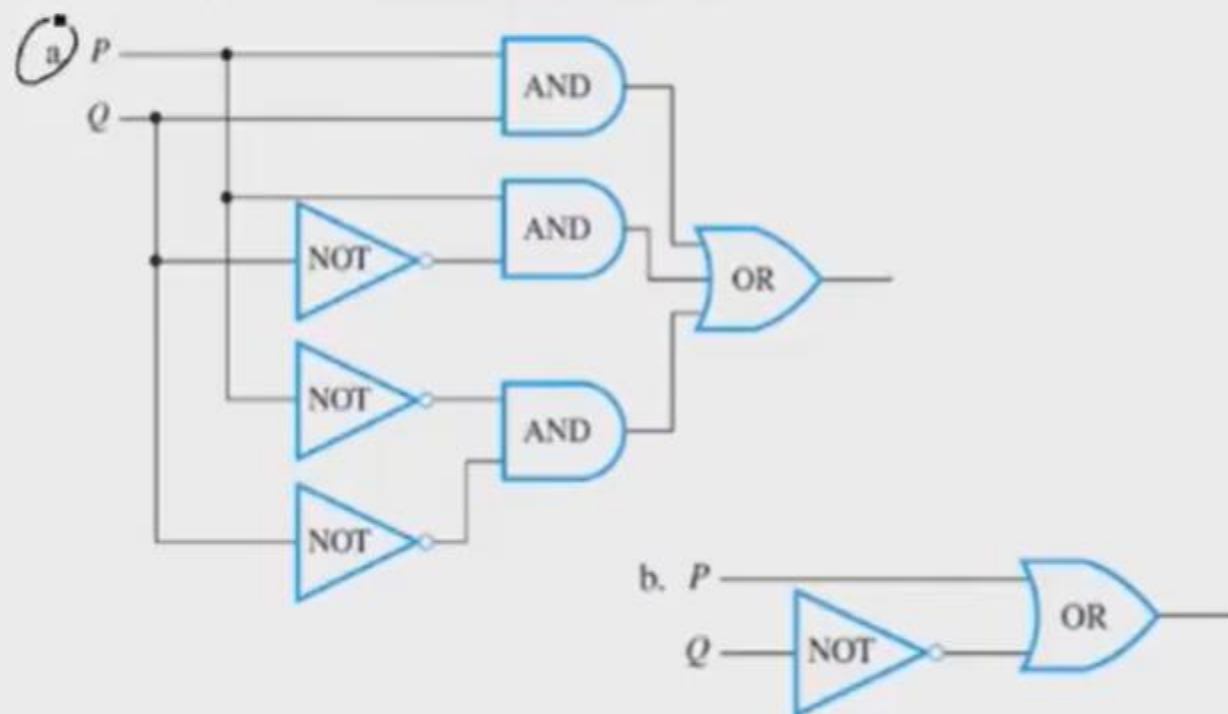
Equivalent Circuits

Thus the two circuits are logically equivalent.



Equivalent Circuits

Find the Boolean expressions for the circuits and show that they are logically equivalent



Equivalent Circuits

Find the Boolean expressions for the circuits and show that they are logically equivalent

$$\begin{aligned} \text{LHS} &= (P \wedge Q) \vee (P \wedge \sim Q) \vee (\sim P \wedge \sim Q) \\ &\equiv ((P \wedge Q) \vee (P \wedge \sim Q)) \vee (\sim P \wedge \sim Q) && \text{by inserting parentheses (which is legal by the associative law)} \\ &\equiv (P \wedge (Q \vee \sim Q)) \vee (\sim P \wedge \sim Q) && \text{by the distributive law} \\ &\equiv (P \wedge \mathbf{t}) \vee (\sim P \wedge \sim Q) && \text{by the negation law for } \vee \\ &\equiv P \vee (\sim P \wedge \sim Q) && \text{by the identity law for } \wedge \\ &\equiv (P \vee \sim P) \wedge (P \vee \sim Q) && \text{by the distributive law} \\ &\equiv \mathbf{t} \wedge (P \vee \sim Q) && \text{by the negation law for } \vee \\ &\equiv (P \vee \sim Q) \wedge \mathbf{t} && \text{by the commutative law for } \wedge \\ &\equiv P \vee \sim Q && \text{by the identity law for } \wedge \end{aligned}$$

NAND and NOR Gates

Another way to simplify a circuit is to find an equivalent circuit that uses the least number of different kinds of logic gates. Two gates not previously introduced are useful for this: NAND-gate and NOR-gate.

A NAND-gate is a single gate that acts like an AND-gate followed by a NOT-gate.

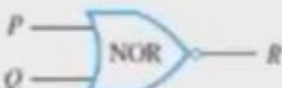
A NOR-gate acts like an OR-gate followed by a NOT-gate.

Thus the output signal of a NAND-gate is 0 when, and only when, both input signals are 1, and the output signal for a NOR-gate is 1 when, and only when, both input signals are 0.

The logical symbols corresponding to these gates are $|$ (for NAND) and $\downarrow$ (for NOR), where $|$ is called a **Sheffer stroke** and $\downarrow$ is called a **Peirce arrow**. Thus

$$P | Q \equiv \sim(P \wedge Q) \quad \text{and} \quad P \downarrow Q \equiv \sim(P \vee Q).$$

NAND and NOR Gates

Type of Gate	Symbolic Representation	Action																		
NAND		<table> <tr> <th colspan="2">Input</th><th>Output</th></tr> <tr> <th>P</th><th>Q</th><th>$R = P \downarrow Q$</th></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>1</td></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> </table>	Input		Output	P	Q	$R = P \downarrow Q$	1	1	0	1	0	1	0	1	1	0	0	1
Input		Output																		
P	Q	$R = P \downarrow Q$																		
1	1	0																		
1	0	1																		
0	1	1																		
0	0	1																		
NOR		<table> <tr> <th colspan="2">Input</th><th>Output</th></tr> <tr> <th>P</th><th>Q</th><th>$R = P \uparrow Q$</th></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>0</td></tr> <tr> <td>0</td><td>1</td><td>0</td></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> </table>	Input		Output	P	Q	$R = P \uparrow Q$	1	1	0	1	0	0	0	1	0	0	0	1
Input		Output																		
P	Q	$R = P \uparrow Q$																		
1	1	0																		
1	0	0																		
0	1	0																		
0	0	1																		

NAND and NOR Gates

It can be shown that any Boolean expression is equivalent to one written entirely with Sheffer strokes or entirely with Peirce arrows. Thus any digital logic circuit is equivalent to one that uses only NAND-gates or only NOR-gates.

NAND and NOR Gates

It can be shown that any Boolean expression is equivalent to one written entirely with Sheffer strokes or entirely with Peirce arrows. Thus any digital logic circuit is equivalent to one that uses only NAND-gates or only NOR-gates.

$$\begin{aligned} \sim P &\equiv \sim(P \wedge P) \equiv P | P \\ P | Q &\equiv \sim(P \wedge Q) \\ P \wedge P &\equiv P \\ \sim(\sim P) &\equiv P \quad \text{negation law.} \\ \sim(\underbrace{P | P}_R \wedge \underbrace{Q | Q}_W) &\equiv \sim(R \wedge W) \end{aligned}$$

Rewriting Expressions Using the Sheffer Stroke

Use the definition of Sheffer stroke to show that

a. $\sim P \equiv P | P$ b. $P \vee Q \equiv (P | P) | (Q | Q)$.

a. $\sim P \equiv \sim(P \wedge P)$ by the idempotent law for $\wedge$
 $\equiv P | P$ by definition of $|$.

b. $P \vee Q \equiv \sim(\sim(P \vee Q))$ by the double negative law
 $\equiv \sim(\sim P \wedge \sim Q)$ by De Morgan's laws
 $\equiv \sim((P | P) \wedge (Q | Q))$ by part (a)
 $\equiv (P | P) | (Q | Q)$ by definition of $|$.

Rewriting Expressions Using the Peirce Arrow

Show that the following logical equivalences hold for the Peirce arrow $\downarrow$, where $\underline{P \downarrow Q} \equiv \sim(P \vee Q)$.

a. $\sim P \equiv P \downarrow P$

b. $P \vee Q \equiv (P \downarrow Q) \downarrow (P \downarrow Q)$

c. $P \wedge Q \equiv (P \downarrow P) \downarrow (Q \downarrow Q)$

b. $(P \downarrow Q) \downarrow (P \downarrow Q)$

$$\equiv \sim(P \downarrow Q) \quad \text{by part (a)}$$

$$\equiv \sim[\sim(P \vee Q)] \quad \text{by definition of } \downarrow$$

$$\equiv P \vee Q \quad \text{by the double negative law}$$

Rewriting Expressions Using the Peirce Arrow

Show that the following logical equivalences hold for the Peirce arrow $\downarrow$, where $P \downarrow Q \equiv \sim(P \vee Q)$.

a. $\sim P \equiv P \downarrow P$

b. $P \vee Q \equiv (P \downarrow Q) \downarrow (P \downarrow Q)$

c. $P \wedge Q \equiv (P \downarrow P) \downarrow (Q \downarrow Q)$

$$\sim P \equiv \sim(P \vee P)$$

$$\equiv P \downarrow P$$

$$P \vee Q \equiv \sim(\sim(P \vee Q))$$

$$\equiv \sim(P \downarrow Q)$$

$$\equiv (P \downarrow Q) \downarrow (P \downarrow Q)$$

b. $(P \downarrow Q) \downarrow (P \downarrow Q)$

$$\equiv \sim(P \downarrow Q)$$

$$\equiv \sim[\sim(P \vee Q)]$$

$$\equiv P \vee Q$$

by part (a)

by definition of $\downarrow$

by the double negative law

Boolean Algebra

Decimal representations

$$\begin{aligned} 6152 &= 6 \cdot 1000 + 1 \cdot 100 + 5 \cdot 10 + 2 \cdot 1 \\ &= 6 \cdot 10^3 + 1 \cdot 10^2 + 5 \cdot 10^1 + 2 \cdot 10^0. \end{aligned}$$

More generally, decimal notation is based on the fact that any positive integer can be written uniquely as a sum of products of the form

$$d \cdot 10^n$$

where each n is a nonnegative integer and each d is one of the decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. The word *decimal* comes from the Latin root *deci*, meaning "ten."

Boolean Algebra

Converting decimal to binary representations



$$\begin{aligned} 27 &= 16 + 8 + 2 + 1 \\ &= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0. \end{aligned}$$

$$\begin{array}{ccccccc} 1 \cdot 2^4 & + & 1 \cdot 2^3 & + & 0 \cdot 2^2 & + & 1 \cdot 2^1 & + & 1 \cdot 2^0 \\ | & & | & & | & & | & & | \\ \hline 27_{10} & & & = & & & 1 & 1 & 0 & 1 & 1_2 \end{array}$$

Power of 2	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Decimal Form	1024	512	256	128	64	32	16	8	4	2	1

Converting binary to decimal

Represent $(110101)_2$ in decimal notation.

$$\begin{aligned} 110101_2 &= 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 16 + 4 + 1 \\ &= 53_{10} \end{aligned}$$

$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	
1	1	0	1	0	1	
						$1 \cdot 1 = 1$
						$0 \cdot 2 = 0$
						$1 \cdot 4 = 4$
						$0 \cdot 8 = 0$
						$1 \cdot 16 = 16$
						$1 \cdot 32 = 32$
						$\underline{53}_{10}$

Addition in Binary Notation

Add 1101_2 and 111_2 using binary notation.

Solution: Because $2_{10} = 10_2$ and $1_{10} = 1_2$, the translation of $1_{10} + 1_{10} = 2_{10}$ to binary notation is

$$\begin{array}{r} 1_2 \\ + 1_2 \\ \hline 10_2 \end{array}$$

It follows that adding two 1's together results in a carry of 1 when binary notation is used. Adding three 1's together also results in a carry of 1 since $3_{10} = 11_2$ ("one one base two").

$$\begin{array}{r} 1_2 \\ + 1_2 \\ + 1_2 \\ \hline 11_2 \end{array}$$

Boolean Algebra

Any integer can be represented uniquely as a sum of products of the form

$$d \cdot 2^n$$

where each n is an integer and each d is one of the binary digits (or bits) 0 or 1.

$1_{10} =$	$1 \cdot 2^0 =$	1_2
$2_{10} =$	$1 \cdot 2^1 + 0 \cdot 2^0 =$	10_2
$3_{10} =$	$1 \cdot 2^1 + 1 \cdot 2^0 =$	11_2
$4_{10} =$	$1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 =$	100_2
$5_{10} =$	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$	101_2
$6_{10} =$	$1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 =$	110_2
$7_{10} =$	$1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 =$	111_2
$8_{10} =$	$1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 =$	1000_2
$9_{10} =$	$1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$	1001_2

Addition in Binary Notation

Thus the addition can be performed as follows:

A hand-drawn diagram illustrating binary addition. It shows two numbers being added: 1101 (top) and 1010 (bottom). The sum is 10102. The diagram highlights the carry propagation from right to left. The rightmost column (1+0) results in 02. The next column (0+1) results in 12. The third column (1+1) results in 12. The leftmost column (1+0) results in 102. The carry is shown as a '1' in a circle at the top right, with arrows indicating its movement from the third column to the fourth, and from the second column to the third. A blue arrow labeled '← carry row' points to the carry '1' in the third column.

$$\begin{array}{r} 1101 \\ + 1010 \\ \hline 1010_2 \end{array}$$

Boolean Algebra

- Just like Boolean logic, variables can only be 1 or 0, instead of true/false

- Not

$$\sim 0 = 1$$

$$\sim 1 = 0$$

- Or is used as a plus

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1+1=?$$

$$2 = 11_2$$

$$10$$

$$\begin{array}{c} T, F \\ \swarrow \searrow \\ 1, 0 \end{array} \quad \begin{array}{l} \sim F = T \\ \sim T = F \end{array}$$

- And is used as a multiplication

$$0 * 0 = 0$$

$$0 * 1 = 0$$

$$1 * 0 = 0$$

$$1 * 1 = 1$$

Half Adder

- Consider adding two 1-bit binary numbers x and y

→ $0+0 = 0$

→ $0+1 = 1$

→ $1+0 = 1$

→ $1+1 = 10_2$

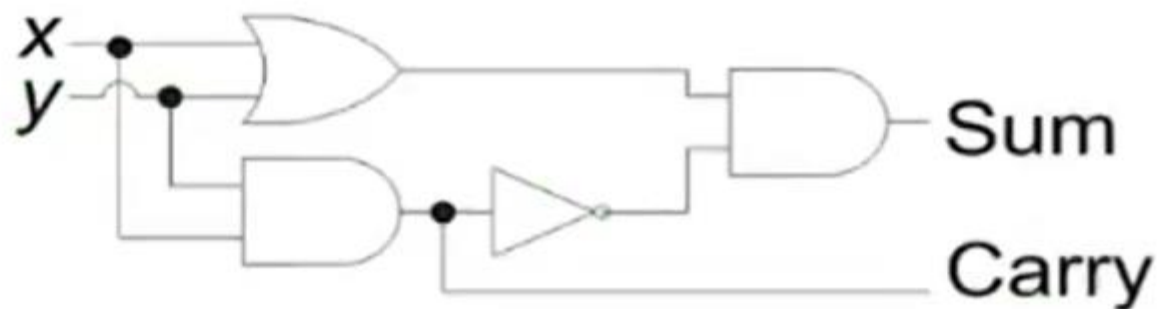
x	y	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- Carry is x AND y
- Sum is x XOR y
- The circuit to compute this is called a half-adder.

Circuit of Half Adder

- Sum = $x \text{ XOR } y$
- Carry = $x \text{ AND } y$

x	y	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Using Half adders

- We can then use a half-adder to compute the sum of two Boolean numbers

$$\begin{array}{r} 1100 \\ + 1110 \\ \hline ?010 \end{array}$$

$$\begin{array}{r} 1100 \\ + 1110 \\ \hline 110 \end{array}$$

How to fix that

- We need to create an adder that can take a carry bit as an additional input

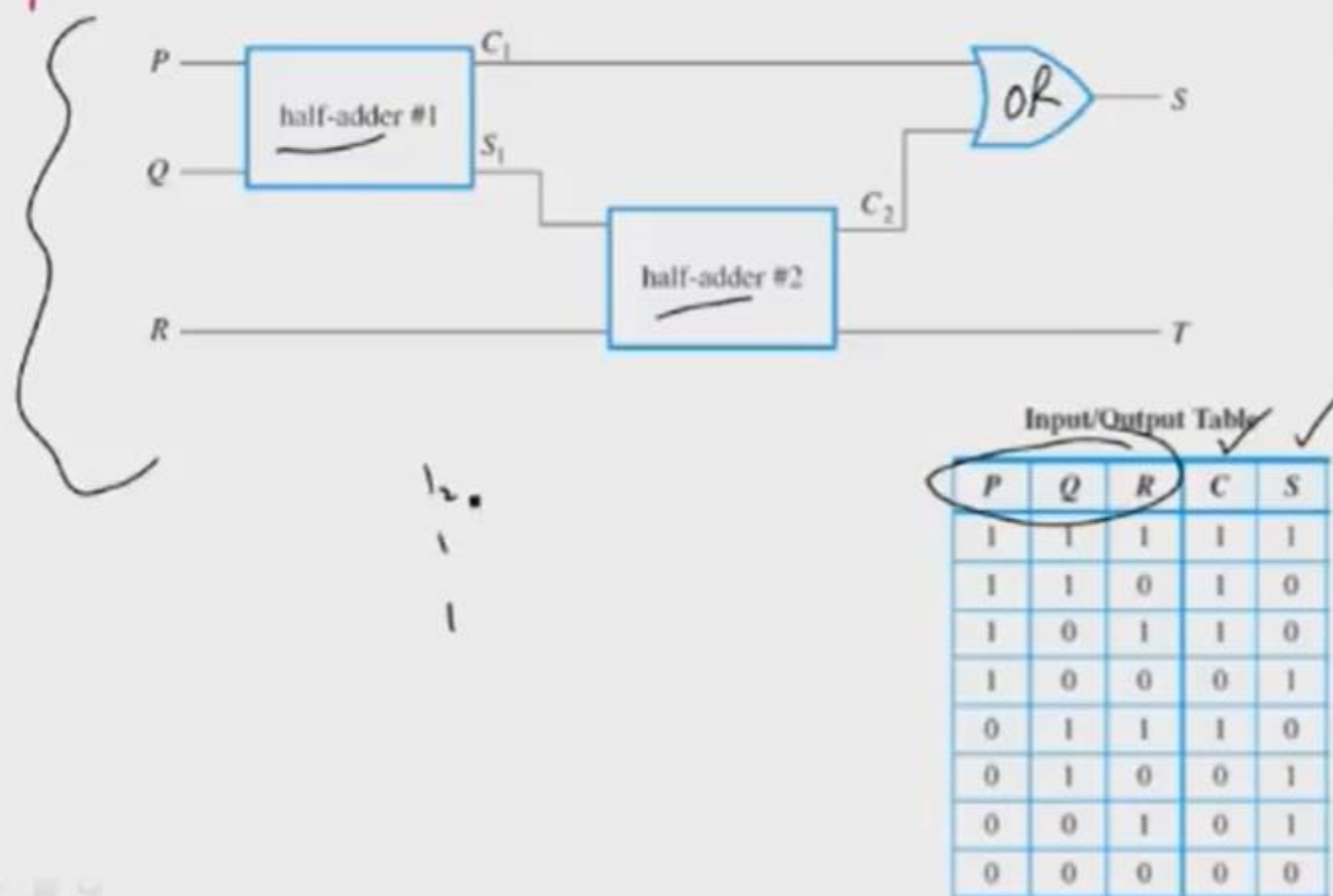
Inputs: x , y , carry in

Outputs: sum, carry out

- This is called a full adder
Will add x and y with a half-adder
Will add the sum of that to the carry in
- What about the carry out?
It's 1 if either (or both):
 $x+y = 10$
 $x+y = 01$ and carry in = 1

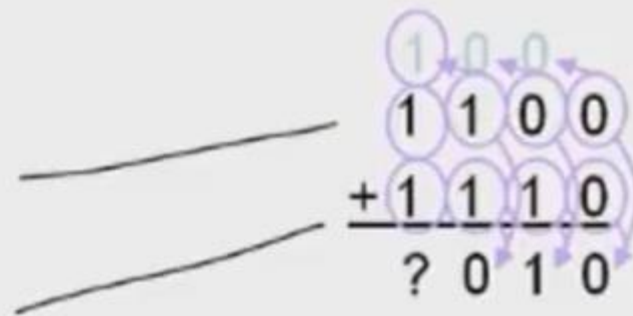
x	y	c	carry	sum
1	1	1	1	1
1	1	0	1	0
1	0	1	1	0
1	0	0	0	1
0	1	1	1	0
0	1	0	0	1
0	0	1	0	1
0	0	0	0	0

The Full adder

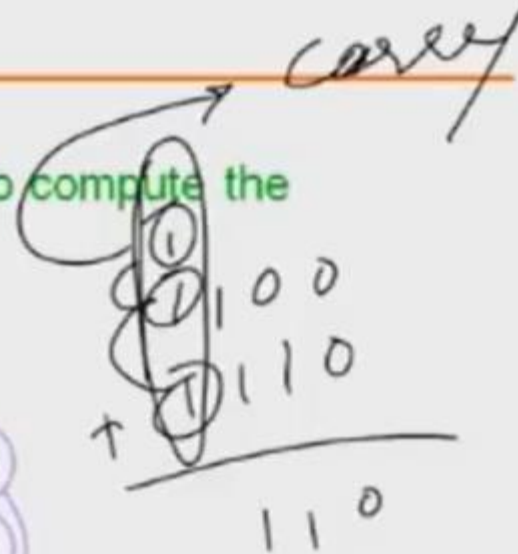


Using Half adders

- We can then use a half-adder to compute the sum of two Boolean numbers


$$\begin{array}{r} 1100 \\ + 1110 \\ \hline ?010 \end{array}$$

The diagram illustrates the propagation of carry bits in a 4-bit addition. The first row is 1100, the second is 1110, and the result is ?010. Purple circles and arrows show the carry chain: a carry of 1 is generated from the rightmost column (0+0), passes through the next column (0+1), and then through the third column (0+1) to the leftmost column (1+1). The carry into the leftmost column is labeled with a green '1'.


$$\begin{array}{r} 100 \\ + 110 \\ \hline 110 \end{array}$$

The diagram illustrates the propagation of carry bits in a 4-bit addition. The first row is 100, the second is 110, and the result is 110. Purple circles and arrows show the carry chain: a carry of 1 is generated from the leftmost column (1+1), passes through the next column (0+1), and then through the third column (0+1) to the rightmost column (0+0). The carry into the rightmost column is labeled with a green '1'.

Parallel Adder Circuits

PQR
 S

Two full-adders and one half-adder can be used together to build a circuit that will add two three-digit binary numbers PQR and STU to obtain the sum $WXYZ$. Such a circuit is called a **parallel adder**. Parallel adders can be constructed to add binary numbers of any finite length.

